Configuring Batch Components

Filtering Records to Process within a Batch Step Component

You can set up and configure Mule batch processing components to perform common batch processing use cases and to enhance the performance of batch jobs for cases in which the defaults require adjustment.

Filtering Records to Process within a Batch Step Component

You can apply one or more record filters as attributes to any number of Batch Step components. For example, assume that the first Batch Step component within a given Batch Job component checks for a Salesforce contact in each record and the second updates the contact information in those records. To ensure that the second Batch Step processes only records that succeeded in the first step, you can configure a filter on the second Batch Step. The filter streamlines processing so that Mule focuses on only the relevant data for a given step.

+ Cache Scope

To filter records, the Batch Step component supports one `acceptExpression` and one `acceptPolicy`. Both are optional, and no other Mule components accept these filters. If you use both filters on the same component, Mule evaluates the filters in the following order:

1. Accept Policy: acceptPolicy
2. Accept Expression: acceptExpression

A Batch Step component that uses an `acceptExpression` attribute applies a DataWeave expression to each record that reaches the component and accepts a record for processing within the component only if the expression evaluates to `true` for that record. If the record evaluates to `false`, the Batch Step component skips the record, and that record becomes available to the next Batch Step component within the Batch Job component if one exists.

The example below filters out all records in which the `age` value is less than `21`:

XML

```
<batch:job jobName="batchJob">
  <batch:process-records >
    <batch:step name="adultsOnlyStep" acceptExpression="#[payload.age > 21]">
      ...
    </batch:step>
  </batch:process-records>
</batch:job>
```

A Batch Step component uses an `acceptPolicy` attribute to apply a policy to records processed in a previous Batch Step component. The Batch Step accepts a record for processing within the component only if the policy evaluates to `true` for that record. The default policy is `NO_FAILURES`. Other policies are `ONLY_FAILURES` and `ALL`. For descriptions of these policies, see Batch Step Component (<batch:step/>).

In the following example, the second Batch Step component (`batchStep2`) accepts only those records that failed to process successfully during the preceding step (`batchStep1`). For example, assume processors within `batchStep1` check each record for a Salesforce contact. If the check fails to find a contact, `batchStep2` accepts only those records without contacts, skipping records with contacts. Message processors within `batchStep2` can add a contact to those records.

XML

```xml
<batch:job jobName="batchJob">
  <batch:process-records >
          <batch:step name="batchStep1" >
      <!-- Check for contact -->
                  ...
    </batch:step>
    <batch:step name="batchStep2" accept-policy="ONLY_FAILURES">
      <!-- Accept records that failed -->
                  ...
    </batch:step>
  </batch:process-records>
</batch:job>
```

For demonstration purposes, the following example uses the Raise Error component (`<raise-error />`) to produce failures in the first Batch Step (`batchStep1`). The second Batch Step (`batchStep2`) accepts failed records only (`ONLY_FAILURES`) and logs the payload of those records. The third uses the default `acceptPolicy` to accept `NO_FAILURES` .

XML

```xml
<flow name="batch-exampleFlow" >
  <scheduler doc:name="Scheduler" >
    <scheduling-strategy >
      <fixed-frequency frequency="75" timeUnit="SECONDS"/>
```

```
  </scheduling-strategy>
</scheduler>
<!-- Set Payload -->
<set-payload value="#[[1,2,3,4,5]]" doc:name="Set Payload" />
<!-- Batch Job Component -->
<batch:job jobName="Batch_Job" maxFailedRecords="-1">
  <batch:process-records >
    <!-- First Batch Step defaults to NO_FAILURES -->
    <batch:step name="batchStep1" acceptPolicy="ALL"
                acceptExpression="#[payload &lt; 3]">
      <!-- Raising Error -->
      <raise-error doc:name="Raise error"
                   type="MY:APP" description="Example: Raising Error "/>
```

Expand content

- The Set Payload component (`<set-payload />`) sends the array `[1,2,3,4,5]` to the Batch Job component.
- The Batch Job component (`<batch:job />`) sets `maxFailedRecords` to `-1` so that failed records within the first Batch Step component do not stop the batch job instance from processing.
- The first Batch Step (`batchStep1`) configures `acceptExpression="#[payload < 3]"` to accept only records with a payload less than `3` and throws an error on those records because of the Raise Error component within it:

```
LOGS |
INFO  ...DefaultBatchStep: Found error processing record on step 'batchStep1' for j
of job 'batch-example-policiesBatch_Job'.
```

- The second Batch Step (`batchStep2`) prints the payload of the failed records, for example:

```
LOGS |
INFO  ... LOGGER IN SECOND BATCH STEP: 1
INFO  ... LOGGER IN SECOND BATCH STEP: 2
```

- The third Batch Step (`batchStep3`) prints the payload of the successful records, for example:

```
LOGS |
INFO ... LOGS IN THIRD BATCH STEP: 3
INFO  ...LOGS IN THIRD BATCH STEP: 4
INFO  ...LOGS IN THIRD BATCH STEP: 5
```

- After the batch job instance completes, the logs indicate that `3` records processed successfully and `2` records failed:

```
LOGS |
INFO  ...engine.DefaultBatchEngine: Finished execution for instance ...
Total Records processed: 5. Successful records: 3. Failed Records: 2
```

For more information, see Handling Errors During Batch Job.

Performing Bulk Operations from a Batch Aggregator Component

Aggregation is useful for sending multiple records within an array to an external server. Within the Batch Aggregator component, you can add an operation, such as a bulk upsert, insert, or update operation, to load multiple records to a server with a single execution of an operation, instead of running an operation separately on each record.

You can process records in separate arrays of a fixed size or stream a single array of records from the batch job instance. For example, you can configure an Upsert operation in the Anypoint Connector for Salesforce (Salesforce Connector) to upsert 200 processed records. Alternatively, you can stream all the records in the instance to the server.

The following example bulk upserts records in separate arrays of 200 records per upsert:

XML

```xml
<batch:job jobName="batchJob">
  <batch:process-records >
    <batch:step name="batchStep">
      <batch:aggregator size="200">
        <salesforce:upsert doc:name="Upsert" ... />
      </batch:aggregator>
    </batch:step>
  </batch:process-records>
</batch:job>
```

The following example streams upserts to a database:

XML

```xml
<batch:job jobName="batchJob">
        <batch:process-records >
                <batch:step name="batchStep">
                  <batch:aggregator streaming="true">
```

```
                    salesforce:upsert doc:name="Upsert" ... />
                </batch:aggregator>
            </batch:step>
        </batch:process-records>
</batch:job>
```

Error handling:
Some connectors handle record-level errors without causing an entire batch aggregation process to fail, for example, Anypoint Connector for Salesforce (Salesforce Connector) and Anypoint Connector for NetSuite (NetSuite Connector). At runtime, these connectors keep track of records that the target resource accepts successfully and which fail. Rather than failing the entire group of records, the connector upserts as many records as it can and tracks any failures for notification purposes.

To learn more about processing within the Batch Aggregator component and how aggregation differs from processing within the Batch Step component, see Process Phase.

Modifying Records within a Batch Aggregator

You can modify records within the Batch Aggregator component, just as you can modify them with processors in the Batch Step component. The modifications can take place sequentially or through random access to specific records.

Sequential Processing within a Batch Aggregator

The following example performs sequential access to records. Using Groovy within a Scripting module, the example modifies the payload of each record from the For Each component's iteration and creates a variable for each collected record.

XML

```
<batch:job jobName="batchJob">
  <batch:process-records>
    <batch:step name="batchStep">
      <batch:aggregator doc:name="batchAggregator" size="10">
        <foreach doc:name="For Each">
          <script:execute engine="groovy">
            <script:code>
              vars['marco'] = 'polo'
              vars['record'].payload = 'hello'
            </script:code>
          </script:execute>
        </foreach>
      </batch:aggregator>
    </batch:step>
  </batch:process-records>
</batch:job>
```

Randomly Accessing Records within a Batch Aggregator

You can also use the For Each scope when randomly accessing records by their iteration number. For Each exposes a `records` variable, which is an immutable list that it uses to keep track of the iteration. You can use this variable to randomly access records in the list from the Batch Aggregator component.

To demonstrate random access when using fixed-size aggregation, the following example specifies the index of a record in the For Each list of records. Instead of sequentially accessing each record, using the `records` variable with an index of a record selects a single record from the list. The example uses the Scripting module to modify the payload of the selected record and create a variable for that record:

XML

```xml
<batch:job jobName="batchJob">
  <batch:process-records>
    <batch:step name="batchStep">
      <batch:aggregator doc:name="batchAggregator" size="10">
        <foreach doc:name="For Each">
          <script:execute engine="groovy">
            <script:code>
              records[0].vars['marco'] = 'polo'
              records[0].vars['record'].payload = 'hello'
            </script:code>
          </script:execute>
        </foreach>
      </batch:aggregator>
    </batch:step>
  </batch:process-records>
</batch:job>
```

You can configure a Batch Aggregator component to stream its content. Setting this component to stream the records (`streaming="true"`) enables you to process an array of all the records in the batch job instance without running out of memory, regardless of how many or how large the records are. For example, if you need to

write millions of records to a CSV file, you can stream the records with the Batch Aggregator component.

```xml
<batch:job jobName="batchJob">
  <batch:process-records >
    <batch:step name="batchStep">
      <batch:aggregator streaming="true">
        <file:write path="reallyLarge.csv">
          <file:content><![CDATA[%dw 2.0
            ...

          }]]></file:content>
      </batch:aggregator>
    </batch:step>
  </batch:process-records>
</batch:job>
```

As the following example shows, only sequential access to records is possible when streaming within the Batch Aggregator component. Random access is not supported when streaming because streaming requires access to the entire set of records. This restriction is necessary because Mule cannot guarantee that all records will fit in memory if no fixed size is specified.

```xml
<batch:job jobName="batchJob">
  <batch:process-records>
    <batch:step name="batchStep">
      <batch:aggregator doc:name="batchAggregator" streaming="true">
        <foreach doc:name="For Each">
          <script:execute engine="groovy">
            <script:code>
              vars['marco'] = 'polo'
```

```
            vars['record'].payload = 'foo'
          </script:code>
        </script:execute>
      </foreach>
    </batch:aggregator>
  </batch:step>
 </batch:process-records>
</batch:job>
```

To learn more about processing within the Batch Aggregator component and how aggregation differs from processing within the Batch Step component, see Process Phase.

NOTE

When using a Batch Aggregator component, do not attempt to process Guava data types, such as `ImmutableMap`. Instead, use a Java Map to avoid serialization issues.

Preserving the MIME types of the Aggregated Records

Introduced in Mule 4.3

Aggregated records pass into the Batch Aggregator component as an array containing each record's payload. However, by default, the MIME types associated with those payloads are not preserved. To preserve record's MIME types, specify the `preserveMimeTypes` attribute in the Batch Aggregator component.

Consider the following JSON array:

JSON

```
[
  {
    "name": "Tony Stark",
    "alias": "Iron Man",
    "status": "DEAD"
  },
  {
    "name": "Steve Rodgers",
    "alias": "Captain America",
    "status": "RETIRED"
  },
  {
    "name": "Peter Parker",
    "alias": "SpiderMan",
    "status": "FUGITIVE"
  }
]
```

Assume that the JSON array is the input to the following Batch Job component:

XML

```
<batch:job name="avengersLogger">
  <batch:process-records>
    <batch:step name="log">
      <batch:aggregator size="10">
        <foreach>
          <logger message="Agent #[payload.alias] is #[payload.status]" />
        </foreach>
      </batch:aggregator>
    </batch:step>
  </batch:process-records>
</batch:job>
```

The batch engine splits the input JSON array into individual records, which aggregator block receives an array with three elements. The first one of them is:

```json
{
  "name": "Tony Stark",
  "alias": "Iron Man",
  "status": "DEAD"
}
```

However, when the Logger component attempts to evaluate the `#[payload.alias]` expression, an error similar to the following one results:

```
********************************************************************************
Message               : "You called the function 'Value Selector' with these argument
  1: Binary ("ewogICJmaXJzdE5hbWUiOiAiUmFtIiwKICAibGFzdE5hbWUiOiAiUmFtMSIsCiAgImFkZHJ
  2: Name ("alias")

But it expects one of these combinations:
  (Array, Name)
  (Array, String)
  (Date, Name)
  (DateTime, Name)
  (LocalDateTime, Name)
  (LocalTime, Name)
  (Object, Name)
  (Object, String)
  (Period, Name)
  (Time, Name)
```

```
5|                                        name: payload.alias,
                                               ^^^^^^^^^^^^^
```

This error occurs because MIME types are not preserved, which prevents Mule from reading it as JSON. To resolve this issue, you must set the `preserveMimeTypes` attribute in the Batch Aggregator component to `true`, for example:

XML

```xml
<batch:aggregator size="10" preserveMimeTypes="true">
  <foreach>
    <logger message="Agent #[payload.alias] is #[payload.status]" />
  </foreach>
</batch:aggregator>
```

The setting makes Mule maintain each record's media type and treat the payload as a JSON document.

Changing the Record Block Size

A traditional online processing model typically maps each request to a worker thread. Generally, there is 1:1 relationship between a request and a running thread regardless of the processing type. This relationship holds for synchronous, asynchronous, one-way, and request-response processing types, and when temporarily buffering requests before processing.

However, in a batch job, the traditional threading model does not apply because records are first stored in a persistent queue, before the Process phase begins.

To improve performance, Mule runtime queues and schedules batch records in blocks of up to 100 records per thread. This behavior reduces the number of I/O requests and improves an operation's load. Batch jobs use Mule thread pools, so there is no default for the job. Each thread iterates through that block to process each record, and then each block is queued back, and the process continues.

Consider having 1 million records to place in a queue for a 3-step batch job. At least three million I/O operations occur as the Mule runtime engine takes and requests each record as they move through the job's phases.
Performance requires having enough available memory to process the threads in parallel, which means moving the records from persistent storage into RAM. The larger your records and their quantity, the more available memory you need for batch processing.

Although the standard model of up to 100 records per thread in the batch job works for most use cases, consider three use cases where you need to increase or decrease the block size:

- Assume you have 200 records to process through a batch job. With the default 100-record block size, Mule can only process two records in parallel at a time. If a batch job has fewer than 101 records to process, then processing becomes sequential. If you need to process heavy payloads, then queueing a hundred records demands a large amount of working memory.
- Consider a batch job that needs to process images, and an average image size of 3 MB. In this case, Mule processes 100-record blocks with payloads of 3 MB in each thread. Hence, your default threading-profile setting would require a large amount of working memory just to keep the blocks in the queue. In this case, set a lower block size to distribute each payload through more jobs and lessen the load on your available memory.
- Suppose you have 5 million records with payloads so small that you can fit blocks of 500 records in your memory without problems. Setting a larger block size improves your batch job time without sacrificing working memory load.

To take full advantage of this feature, you must understand how the block sizes affect your batch job. Running comparative tests with different values and testing performance helps you find an optimum block size before moving this change into production.

Remember that modifying the batch block size is optional. If you apply no changes, the default value is 100 records per block.

You set the size through the Batch Job component, for example:

```xml
<batch:job jobName="atch_Job" blockSize="200">
  ...
</batch:job>
```

Setting a Max Concurrency Limit on Batch Job Instances

The Max Concurrency (`maxConcurrency`) property limits the number of record blocks to process concurrently.

You can configure the `maxConcurrency` property as in the following example:

```xml
<batch:job jobName="test-batch" maxConcurrency="${batch.max.concurrency}">
  ...
</batch:job>
```

By default, the Batch Job component limits the maximum concurrency to twice the number of available cores. The capacity of the system running the Mule instance also limits concurrency.

See Also

- Batch Processing
- Batch Job Instance ID
- Back-Pressure and MaxConcurrency

Back to top

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

Yes No

Legal | Terms of Service | Privacy | Trust | Contact | Responsible Disclosure | Cookie Settings
Your Privacy Choices

© Copyright 2026 Salesforce, Inc. All rights reserved. Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States